Project Report: Game Collection

Course: PGR211 Advanced Programming for Data Science Submission: Mandatory mini project
Projects included: Two Hangman implementations and one Tic-Tac-Toe implementation

---

1. Project Overview

This repository contains three separate game projects developed for the mandatory mini project assignment. The collection includes two different implementations of a Hangman-style word guessing game and one implementation of a Tic-Tac-Toe simulator.

Each project demonstrates Python programming concepts through a playable terminal-based game. The projects also explore object-oriented design, user input handling, game state management, and different ways of running or serving command-line applications.

The three included projects are:

Row 1
Project: Arnout Opfergelt Hangman
Type: Word guessing game
Main Focus: Simple procedural-style Hangman implementation

Row 2
Project: Yorbe Heeren Hangman
Type: Word guessing game
Main Focus: Object-oriented Hangman implementation with abstractions

Row 3
Project: Martin Gnecco Tic-Tac-Toe
Type: Tic-Tac-Toe simulator
Main Focus: Object-oriented board game simulation with Rich terminal output

---

2. Repository Structure

```
Python_Mini_Project/
|-- .gitignore
|-- Arnout_Opfergelt_Hangman/
|   |-- README.md
|   ‘-- word_guessing_game.py
|-- Yorbe_Heeren_Hangman/
|   |-- README.md
|   |-- Mini_Project_1.ipynb
|   ‘-- word_guessing_game.py
|-- Martin_Gnecco/
|   |-- README.md
|   |-- main.py
|   |-- playground.py
|   |-- pyproject.toml
|   |-- uv.lock
|   ‘-- ssh_server/
|       |-- go.mod
|       |-- go.sum
|       ‘-- main.go
‘-- README.md
```

The games can also be accessed remotely from a terminal through SSH:

```
ssh -p 23234 terminal.martingneccoh.io # Tic-Tac-Toe
ssh -p 23235 terminal.martingneccoh.io # Hangman v1
```

```
ssh -p 23236 terminal.martingneccoh.io # Hangman v2
```

---

3. Project 1: Arnout Opfergelt Hangman

3.1 Overview

This project is a simple Python word guessing game. The player guesses one letter at a time and can make up to twelve wrong guesses before losing.

The game selects a hidden word, displays unknown letters as underscores, and reveals letters as the player guesses them correctly. The round ends when the player reveals every letter or runs out of turns.

3.2 Requirements

The project requires:

- Python 3
- The wordfreq library

Install the required library with:

```
pip install wordfreq
```

3.3 Running the Game

Open a terminal in the folder containing word_guessing_game.py and run:

```
python word_guessing_game.py
```

3.4 Gameplay

The player follows this flow:

1. Enter a name.
2. Choose a word list.
3. Guess one letter at a time.
4. Correct guesses reveal letters in the word.
5. Wrong guesses reduce the remaining turns.
6. The player wins by revealing every letter before the turns run out.

The word list options are:

- selected words: picks from science, python, rainbow, computer, and programming.
- random words: picks from the 50,000 most common English entries provided by wordfreq.

If the player enters an invalid word-list choice, the game defaults to a random word.

3.5 Code Explanation

The program uses top_n_list("en", 50_000) from wordfreq to get a large list of English words. A word is then selected using random.choice().

The main state variables are:

Row 1
Variable: word
Purpose: Stores the secret word

Row 2
Variable: guesses
Purpose: Stores letters already guessed by the player

Row 3
Variable: turns
Purpose: Starts at 12 and decreases after each wrong guess

A while loop keeps the game running while turns remain. Inside the loop, a for loop checks each letter in the secret word. Guessed letters are displayed, while unknown letters are shown as underscores.

The missing variable counts letters that are still hidden. When no letters are missing, the game prints You Win! and ends the loop.

Input is cleaned by stripping surrounding spaces and converting guesses to lowercase. Only one alphabetic character is accepted at a time. Invalid input and repeated guesses do not cost a turn.

3.6 Current Behaviour

The current version prints the answer before the guessing starts:

```
print(word, " ")
```

This line should be removed if the answer should remain hidden during the game.

---

4. Project 2: Yorbe Heeren Hangman

4.1 Overview

This project is also a word guessing game, but it uses a more structured object-oriented design. A secret word is selected from a list, the player guesses one letter at a time, and correct letters are revealed in their correct positions.

Wrong guesses cost one of the twelve available turns. The round ends when every letter has been guessed or the player runs out of turns.

The implementation focuses on the course learning outcomes related to classes, objects, methods, inheritance, polymorphism, encapsulation, and abstraction.

4.2 Structure

Row 1
Layer: Word selection
Types: WordSource, FixedWordSource, RandomWordSource, DifficultyWordSource
Responsibility: Decide which word is played

Row 2
Layer: Game rules
Types: WordGuessingGame, GuessOutcome, GameState
Responsibility: Hold the state and enforce the rules

Row 3
Layer: Presentation
Types: ConsoleGameRunner, main
Responsibility: Read from and write to the console

4.3 Learning Outcome Coverage

Row 1
Concept: Class and object
Demonstration: WordGuessingGame, ConsoleGameRunner, and the word source classes each model separate responsibilities

Row 2
Concept: Method

Demonstration: guess(), masked_word(), reveal_word(), run(), and next_word() define behavior

Row 3
Concept: Inheritance
Demonstration: FixedWordSource and RandomWordSource derive from WordSource; DifficultyWordSource derives from RandomWordSource

Row 4
Concept: Polymorphism
Demonstration: WordGuessingGame depends on the WordSource abstraction and calls next_word() without depending on a concrete class

Row 5
Concept: Encapsulation
Demonstration: Private state, read-only properties, immutable copies, controlled mutation, and protected answer disclosure

Row 6
Concept: Abstraction
Demonstration: WordSource is an abstract base class with an abstract method and shared validation

4.4 Edge Cases Handled

The implementation handles several important edge cases:

- Empty word lists, blank words, and words containing spaces, digits, or punctuation are rejected.
- A turn limit below one is rejected.
- Multi-character input, digits, and punctuation are reported as invalid and do not cost a turn.
- Repeated guesses are reported and do not cost a turn.
- Guesses are case-insensitive and surrounding whitespace is ignored.
- Winning on the final turn is possible because the state is recomputed after every guess.
- The win condition is checked before the loss condition.
- Guessing after the round has ended raises an error instead of silently doing nothing.
- A blank player name falls back to a default value.

4.5 AI Use Disclosure

AI was used to help structure and organize the code, add comments, and explain how the game addresses the learning outcomes.

The author wrote and remains responsible for the code. AI was used as a supporting tool to improve readability, presentation, and quality, as well as to help resolve problems and explore potential improvements. The implementation and final decisions remained the author's responsibility.

4.6 Running the Game

Run the script with:

```
python word_guessing_game.py
```

In the notebook version, set PLAY_INTERACTIVELY = True in the entry point cell and run it.

---

5. Project 3: Martin Gnecco Tic-Tac-Toe

5.1 Overview

This project is a simple Tic-Tac-Toe simulator. Two players take turns and randomly choose an empty place on the board.

Whenever a player gets three contiguous marks, that player earns one point. If the board fills up before either player completes a line, the round ends in a tie and nobody gets a point.

To win the full game, a player must reach a target number of points. As soon as a player reaches that goal, the game stops. There is also a maximum number of rounds. If neither player reaches the point goal before the round limit, the game stops and the player with the most points wins. If both players have the same score, the full game is tied.

5.2 Running the Game Locally

This project uses uv to manage the Python environment and dependencies. After cloning or downloading the repository, install the dependencies with:

```
uv sync
```

Then run the game with:

```
uv run python main.py
```

The game can also be played remotely from a terminal with:

```
ssh -p 23234 terminal.martingneccoh.io
```

When connecting for the first time, SSH may warn that the host is not in the known_hosts file. Typing yes adds the server host key to the local machine. No public key copy, username, or shell access is required.

5.3 Libraries Used

Row 1
Library: NumPy
Purpose: Models the board as a two-dimensional array and helps check for winners efficiently

Row 2
Library: Rich
Purpose: Creates improved terminal output, including clearing the screen, aligning text, and printing tables

Row 3
Library: Wish
Purpose: Serves the terminal application over SSH using a Go-based SSH server

The SSH server listens on a specific port and immediately launches the Python game when a client connects. It terminates the connection when the program exits and does not provide shell access.

5.4 Code Design

The code follows an object-oriented programming approach. This makes it easier to model the main entities in the game: the player, the board, and the game itself.

This design gives each entity clear responsibilities. It also makes each object’s state and behavior easier to develop, test, and integrate with the rest of the program.

5.5 Main Classes

Player

The Player class represents one player in the game.

Row 1
Attribute or Method: symbol
Purpose: Holds the player’s mark on the board

Row 2
Attribute or Method: points
Purpose: Holds the player’s score

Row 3
Attribute or Method: name
Purpose: Holds the player’s display name

Row 4
Attribute or Method: play_turn()
Purpose: Chooses a random empty square

Row 5
Attribute or Method: add_point()
Purpose: Adds one point to the player

Board

The Board class represents the Tic-Tac-Toe board.

Row 1
Attribute or Method: size
Purpose: Holds the board size

Row 2
Attribute or Method: squares
Purpose: Holds the current board marks

Row 3
Attribute or Method: empty
Purpose: Holds the available empty squares

Row 4
Attribute or Method: show_possibilities()
Purpose: Finds the empty squares

Row 5
Attribute or Method: check_row()
Purpose: Checks for a winning row

Row 6
Attribute or Method: check_col()
Purpose: Checks for a winning column

Row 7
Attribute or Method: check_diag()
Purpose: Checks for a winning diagonal

Row 8
Attribute or Method: check_winner()
Purpose: Combines the row, column, and diagonal checks

Row 9
Attribute or Method: reset()
Purpose: Clears the board for a new round

Row 10
Attribute or Method: __str__()
Purpose: Formats the board for display

Game

The Game class controls the full game flow.

Row 1
Attribute or Method: board
Purpose: Holds the game board

Row 2
Attribute or Method: players
Purpose: Holds both players

Row 3
Attribute or Method: console
Purpose: Interacts with Rich

Row 4
Attribute or Method: max_points
Purpose: Sets the points needed to win

Row 5
Attribute or Method: max_rounds
Purpose: Sets the round limit

Row 6
Attribute or Method: rounds_played
Purpose: Tracks whether the game should stop

Row 7
Attribute or Method: enter_full_screen()
Purpose: Switches to full screen mode

Row 8
Attribute or Method: leave_full_screen()
Purpose: Exits full screen mode

Row 9
Attribute or Method: printc()
Purpose: Prints centered text with Rich

Row 10
Attribute or Method: play_round()
Purpose: Plays one round

Row 11
Attribute or Method: check_player_points()
Purpose: Gets the highest player score

Row 12
Attribute or Method: leaderboard()
Purpose: Creates a Rich table for the scores

Row 13
Attribute or Method: game_over_reason()
Purpose: Explains why the game stopped

Row 14
Attribute or Method: get_winner()
Purpose: Determines which player won

Row 15
Attribute or Method: winner_message()
Purpose: Creates the final winner message

Row 16
Attribute or Method: play_game()
Purpose: Runs the full game

5.6 Running the SSH Server

The SSH server is optional and is only needed to serve the game through SSH. The server code is inside ssh_server/ and uses Wish to start the Python game when a client connects.

To avoid publishing local machine paths, the server reads its configuration from a local .env file. This file should not be committed because it contains machine-specific paths such as the Python interpreter, the game file, and the project folder.

Create a .env file inside ssh_server/ with values similar to:

```
PYTHON_PATH=/absolute/path/to/.venv/bin/python
GAME_PATH=/absolute/path/to/main.py
PROJECT_PATH=/absolute/path/to/tic_tac_toe
HOST=0.0.0.0
PORT=23234
```

Update the values so they match the local machine before starting the server.

The server expects a host key in the same directory. This key identifies the server and

should not be committed to the repository. Wish can automatically manage the creation of a new host key if one does not already exist.

To start the server locally, run:

```
cd ssh_server
go run .
```

With the example values above, the server listens on port 23234 and can be tested with:

```
ssh -p 23234 localhost
```

---

6. Comparison of the Three Projects

Row 1
Area: Game type
Arnout Hangman: Word guessing
Yorbe Hangman: Word guessing
Martin Tic-Tac-Toe: Board game simulation

Row 2
Area: Main language
Arnout Hangman: Python
Yorbe Hangman: Python
Martin Tic-Tac-Toe: Python, with Go for SSH serving

Row 3
Area: Main style
Arnout Hangman: Simple script-based game loop
Yorbe Hangman: Object-oriented architecture
Martin Tic-Tac-Toe: Object-oriented architecture

Row 4
Area: External libraries
Arnout Hangman: wordfreq
Yorbe Hangman: Standard Python features and notebook support
Martin Tic-Tac-Toe: NumPy, Rich, Wish

Row 5
Area: User interaction
Arnout Hangman: Console input
Yorbe Hangman: Console input or notebook interaction
Martin Tic-Tac-Toe: Automated player turns with terminal display

Row 6
Area: Remote play
Arnout Hangman: Available through repository SSH setup
Yorbe Hangman: Available through repository SSH setup
Martin Tic-Tac-Toe: Available through SSH server

---

7. Conclusion

The three projects show different ways to design and run small terminal games. The first Hangman project focuses on a straightforward implementation that is easy to follow. The second Hangman project expands the same game idea into a more formal object-oriented structure that demonstrates course learning outcomes such as abstraction, inheritance, polymorphism, and encapsulation. The Tic-Tac-Toe project applies object-oriented design to a board game simulation and adds richer terminal presentation and optional SSH-based access.

Together, the projects demonstrate core programming techniques such as loops, conditionals, state management, input validation, class design, dependency usage, and

command-line execution.